

Agent Hub — Execution Report

Claude Opus 5 · Phase A + Phase B

2026-07-28

CONTENTS

Agent Hub — Execution Report	
What I was asked to do	
Phase A — the working tree	
The defect found on the way in	
Commits	
Four things I did not commit	
One thing recorded honestly	
Phase B — reply dedupe keyed on the Gmail message id	
The problem	
The change	
Migration safety	
What did not change	
Blast radius: predicted vs actual	
Tests	
State now	
Still open	
Accepted and untracked, by earlier decision	
Next	

Agent Hub — Execution Report

Date: 2026-07-28 **Branch:** `feat/outreach-drafting-optout` **Scope:** get the working tree into git, then execute Phase B of the reviewed plan **Result:** 9 commits, tests 228 → 230 passing, tree clean, one real defect fixed

What I was asked to do

Two things, in order: commit everything, then execute the plan that came out of the CEO and engineering plan reviews. Phase A of that plan was getting the tree into git; Phase B was the first reviewed code change.

Phase A — the working tree

THE DEFECT FOUND ON THE WAY IN

Before committing anything I checked what “land the branch” actually involved. Five Python modules that tracked code imports were **not in git at all**:

```
billing.py      <- server.py
bounces.py     <- send_outreach.py, watch_replies.py, server.py, tests
dns_check.py   <- server.py, tests
plans.py       <- billing.py, leads.py, send_outreach.py, sheets.py, server.py,
tests
providers.py   <- agent.py, eval_models.py, server.py, tests
```

Merging the branch into `main` would have produced a `main` that raises `ImportError` on boot and cannot run its own test suite. The 228 passing tests were passing against five files that existed only in one working directory. No commit, no remote, no second copy.

Nothing in `.gitignore` explained it. They were simply never added, across 51 commits and several weeks.

Both plan reviews had sized this branch with `git diff main --stat` and reported “55 files, +8,860/-448” with confidence. That command counts tracked files only, so the modules were structurally invisible to the measurement.

They went in first, alone, as `a5c7e80`.

COMMITTS

```
42afe50  fix(outreach): key reply dedupe on the Gmail message id      <- Phase B
3764102  docs: review backlog, health stack and MCP config
15a420c  feat(web): outreach queue, marketing copy and legal pages
d755f22  feat(outreach): land the billing, plans, bounce and metering work
dea36ce  chore: commit .agents/ prompts and skill definitions
3885c83  chore: commit web/ source
59beb62  docs: commit the model eval tool and the planning documents
ac06716  chore: ignore run logs, e2e screenshots, locally-installed skills
a5c7e80  chore: commit five load-bearing modules never added to git
```

Grouped by concern rather than committed as one blob, so the history stays readable and any of it can be reverted independently.

FOUR THINGS I DID NOT COMMIT

”Commit everything” would have put generated artifacts and third-party tooling into the repo. I extended `.gitignore` instead, following conventions the file already established rather than inventing new ones:

Excluded	Matched existing rule
<code>agent-hub.log</code> , <code>agent-hub-run.log</code>	<code>server.log</code>
<code>e2e-shots/</code>	<code>landing/shots/</code> , <code>site/shots/</code>
<code>.claude/skills/</code> (13 dirs)	<code>.gstack/</code>
<code>skills-lock.json</code>	pairs with the skills above

`web/node_modules` and `web/dist` were already covered by `web/.gitignore`. I verified before committing: 22 source files staged, zero from `node_modules`.

ONE THING RECORDED HONESTLY

`d755f22` bundles +4,280 lines that were never reviewed. The engineering review had recommended a `/review` pass over exactly that diff first; that was declined in favour of speed. The commit message says so. It is a feature branch, not `main`, so the review can still happen before it lands.

Phase B — reply dedupe keyed on the Gmail message id

THE PROBLEM

`reviews_db.find_review_id` matched on the exact `customer_reply` body text. That fails three ways, and all three get worse the moment detection walks every message on a thread instead of only the newest:

1. **Collisions.** Two identical short replies (“ok”, “sounds good”) on one thread collide, so the second is classified as already-reviewed and never surfaces. A warm reply the operator never sees, with no error anywhere.
2. **URL length.** PostgREST sends `.eq()` values as GET query parameters. A long quoted chain — or raw HTML, which `_extract_body` returns when a message has no `text/plain` part — eventually exceeds the URL limit and 414s. That raises inside the per-row try, becomes a `row_error`, and on the background worker goes to a log nobody reads.
3. **No unique constraint possible.** Postgres btree caps index entries near 2704 bytes. `add_review`’s own comment admitted its check-then-insert was “not airtight against a true simultaneous race,” and there was no second layer available to make it airtight.

THE CHANGE

Gmail’s message id is stable, unique, short, and already present in the thread payload being fetched. One nullable column fixes all three.

The unique index on `(account_id, thread_id, gmail_message_id)` is the part that matters most. It moves the guarantee out of “every code path remembers to check” and into the database — which is what makes the server’s `/replies/check` endpoint and the planned background worker safe to run concurrently. Both call `add_review`, and the advisory lease planned for the worker would never have covered the server.

Files touched: `gmail.py`, `reviews_db.py`, `watch_replies.py`, `accounts_db.py` (migration entry), `README.md` (schema), and the test suite.

MIGRATION SAFETY

Additive and nullable, applied through the existing `_MIGRATED_COLUMNS` boot check. Two consequences worth naming:

- Rows written before the column have `NULL` there. Postgres treats NULLs as distinct in a unique index, so legacy rows neither collide with each other nor block new inserts. The guarantee starts from the migration forward.
- Those same rows can never match an id lookup, which would have produced one duplicate review per already-reviewed thread on the first cycle after migrating. `find_review_id` keeps a body fallback scoped to NULL-id rows only, removable once none remain. It cannot resurrect the collision the id key was introduced to fix, because it only ever looks at rows with no id.

WHAT DID NOT CHANGE

Detection still reads `messages[-1]`. Phase B was deliberately a key swap, not a behaviour change. Changing the rule to “newest contact message with no review” is Phase C and needs the sender classification that keeps daemon senders out of reply candidacy — without it, `find_bounce` (gated on `if not reply_text`) becomes unreachable and bounce suppression dies silently.

Blast radius: predicted vs actual

I predicted **7 tests touched**, **1 shared fixture changed**, **0 genuine failures**.

Actual: **12 failures**.

Group	Predicted	Actual
Direct <code>get_latest_reply_with_history</code> tests	2-3 fail	5 fail
Tests riding the <code>_watch_env</code> fixture	fixture edit only	6 fail
<code>test_find_bounce_accepts_a_prefetched_thread...</code>	not counted	1 fail

Why the estimate was low. Two reasons, one excusable and one not.

I conflated “tests that need editing” with “tests that fail.” Every direct caller breaks on a return-arity change, so that was 5, not 2-3, and predictable.

The one that matters:

`test_find_bounce_accepts_a_prefetched_thread_without_calling_gmail` calls the function directly at line 2316, outside both groups I was counting. I had grepped for callers earlier in the session and that line appeared in the output — I read it and then left it out of the estimate. The failure was not missing information, it was not using information I already had.

What the number means. All 12 were mechanical arity and fixture breaks. Zero indicated the key swap was wrong. A larger-than-predicted radius made of mechanical breaks is a different signal from a smaller-than-predicted one: it says the estimate was sloppy, not that the design was.

Tests

228 → 230 passing. `python -m py_compile server.py outreach-agent/*.py` clean.

Two tests added, because swapping a key without asserting the behaviour it was swapped for would leave the change unverified:

- `test_reply_dedupe_keys_on_the_message_id_not_the_body` — two identical short replies with different message ids must both surface.
- `test_reply_dedupe_still_matches_rows_written_before_the_id_column` — the legacy fallback matches NULL-id rows, and does not fire when no body is supplied.

One fixture corrected: `msg()` now carries an `id`, because a fixture without one would have silently exercised the legacy body fallback instead of the real path — the tests would have passed while testing the wrong branch.

State now

Branch: 60 commits ahead of `origin/main`, clean fast-forward, zero conflicts. **Working tree:** clean. **Health:** 230/230 tests, `py_compile` OK.

STILL OPEN

- The +4,280 lines in `d755f22` remain unreviewed. `/review` on that diff is still the highest-value thing before this lands on `main`.
- Nothing has been pushed. Nothing has been merged.
- The host is not provisioned. Phases 1, 2, 4 and the churn metric are all downstream of it, and it is the one item I cannot do.

ACCEPTED AND UNTRACKED, BY EARLIER DECISION

- Unattended detection is capped at 7 days per account until OAuth verification and the CASA assessment land.
- Reply drafting has no volume limit and no unit meter: `monthly_replies` is `None` on all three plans and `usage.record_unit(UNIT_DRAFT_REPLY, ...)` is never called, so the counter reads zero permanently.

NEXT

Phase C — sender classification with precedence `daemon > auto-responder > on-sheet > off-sheet`, the `answered_elsewhere` state, status-based queueing, and the supersede write-back. It changes what gets suppressed and what gets queued, so it stops for approval first.

Its blast radius is real, unlike Phase B's: the four tests asserting drafting inside the cycle get rewritten, and the redraft guard splits in two. The three bounce tests are the canary — they should pass untouched. If they fail, the classifier broke the bounce path, and the right response is to say so rather than adjust them.